

Supplementary Code

1 Package quality check - Table S1

```
library(packagemetrics)
library(dplyr)
library(mice)
library(ImputeRobust)
library(countimp)
mi_packages <- package_list_metrics(c("mice", "ImputeRobust", "countimp"))
glimpse(mi_packages)

Rows: 3
Columns: 18
$ package      <chr> "mice", "ImputeRobust", "countimp"
$ published    <chr> "2024-11-27", "2018-11-30", NA
$ title        <chr> "Multivariate Imputation by Chained Equations", "Ro~
$ depends_count <int> 1, 3, NA
$ suggests_count <int> 16, NA, NA
$ tidyverse_happy <lgl> TRUE, TRUE, NA
$ has_vignette_build <lgl> FALSE, FALSE, NA
$ has_tests      <lgl> TRUE, FALSE, NA
$ reverse_count  <dbl> 67, 0, NA
$ dl_last_month  <dbl> 69079, 282, NA
$ ci            <chr> "NONE", NA, NA
$ test_coverage <chr> "NONE", NA, NA
$ stars         <chr> "457", NA, NA
$ forks         <chr> "109", NA, NA
$ last_commit    <dbl> 2.4, NA, NA
$ last_issue_closed <dbl> NA, NA, NA
$ contributors   <dbl> 34, NA, NA
$ watchers      <lgl> NA, NA, NA
```

2 Multiple imputation for derived covariates

In this section, we impute missing BMI from missing weight and height using the three options illustrated in our manuscript. We use the boys dataset from the *mice* package, which contains data on the growth of Dutch boys, with 748 rows and 9 variables, including BMI, weight, and height, which were missing. More information about the dataset can be found by using the command `?boys`.

2.1 Impute, then transform

We impute any missing height and weight data, and append BMI to the imputed data afterward. BMI is not a predictor in the imputation model in this case.

```
# load the necessary library
library(mice)

# First, remove BMI from the imputation model
set.seed(123)
imp0 <- mice(boys, maxit = 0)
matrix <- imp0$predictorMatrix
matrix[, "bmi"] <- 0

# Second, impute height and weight, then derive BMI
imp <- mice(boys, print = FALSE, predictorMatrix = matrix)
long <- mice::complete(imp, "long", include = TRUE) # include = TRUE appends the
# observed boys dataset into the "long" imputed dataset
long$bmi <- with(long, wgt / (hgt/100)^2)
```

2.2 Transform, then impute

We include BMI in the imputation model, and impute BMI as just another variable here.

```
# Impute height, weight and BMI altogether
set.seed(123)
imp <- mice(boys, print = FALSE)
long <- mice::complete(imp, "long", include = TRUE)
```

2.3 Passive imputation

With this approach, we include BMI in the imputation model and additionally specify dependencies among weight, height, and BMI to ensure their relationship is not distorted.

```
# Specify dependencies among weight, height, and BMI before imputation
set.seed(123)
meth <- make.method(boys)
meth["bmi"] <- "~I(wgt / (hgt/100)^2)"
pred <- make.predictorMatrix(boys)
pred[c("wgt", "hgt"), "bmi"] <- 0
imp.pas <- mice(boys, meth = meth, pred = pred,
               print = FALSE)
long.pas <- mice::complete(imp.pas, "long", include = TRUE)
```

3 Warfarin case study - multiple imputation and median imputation

The code below demonstrates how we performed multiple imputation for the 6.25% missing body weight scenario.

```
# Load libraries -----

library(mice)
library(tidyverse)
library(nlmixr2)
library(devtools)
source_url('https://raw.githubusercontent.com/R-miss-tastic/website/master/static/how-to/generate/amputation.R')

# Load data -----

# set seed for reproducibility
set.seed(42)

warfarin_pk <- read_csv("warfarin_pk.csv")

warfarin_pk_baseline <- warfarin_pk |> filter(TIME == 0)

# center WT around 70kg
warfarin_pk <- warfarin_pk |>
  mutate(lnWT = log(WT / 70))

# Create nlmixr2 model objects -----

## Full model ----
complete.model.war.1cpt.ode <- function(){
  ini({
    tka <- -0.654
    tcl <- -1.98
    tv <- 2.09
    wteff_cl <- 0.695
    wteff_v <- 0.792
    eta.cl ~ 0.0545
    eta.v ~ 0.0279
```

```

prop.err <- 0.000468
add.err <- 1.34
})
model({
  ka <- exp(tka)
  cl <- exp(tcl + wteff_cl * lnWT + eta.cl)
  v <- exp(tv + wteff_v * lnWT + eta.v)
  d/dt(depot) = -ka * depot
  d/dt(cent) = ka * depot - cl/v * cent
  cp = cent/v
  cp ~ prop(prop.err) + add(add.err)
})
}

## Base model ----
base.model.war.1cpt.ode <- function(){
  ini({
    tka <- -0.654
    tcl <- -1.98
    tv <- 2.09
    eta.cl ~ 0.0545
    eta.v ~ 0.0279
    prop.err <- 0.000468
    add.err <- 1.34
  })
  model({
    ka <- exp(tka)
    cl <- exp(tcl + eta.cl)
    v <- exp(tv + eta.v)
    d/dt(depot) = -ka * depot
    d/dt(cent) = ka * depot - cl/v * cent
    cp = cent/v
    cp ~ prop(prop.err) + add(add.err)
  })
}

# Extract PK parameters for multiple imputation -----

# fit the base model to the complete data
base.model.estimate <- nlmixr(base.model.war.1cpt.ode,
  warfarin_pk,
  est = "focei",
  control = list(print = 0),
  foceiControl = list(outerOpt = "bobyqa",
    maxInnerIterations = 500))

# extract unique individual PK parameters
base.individual.estimate <- base.model.estimate$dataMergeFull |>
  na.omit() |>
  select(cl, v) |>
  unique()

# reset row numbers to start from 1
row.names(base.individual.estimate) <- 1:nrow(base.individual.estimate)

# add ID column, bring to the front
base.individual.estimate <- base.individual.estimate |>
  mutate(ID = row_number()) |>
  relocate(ID, .before = cl)

# Create 30 datasets with 6.25% WT missing -----

```

```

## Identify eligible seeds -----
# The goal here is to indentify 30 seeds that produce 6.25% missing WT

# Initialize variables
nseed <- c(1:30) # We want 30 seeds in total
seeds_with_sum_2 <- c() # a vector to save all eligible seeds
counter <- 1 # a variable to identify eligible seeds

# Repeat the process
while (length(seeds_with_sum_2) < length(nseed)) {

  warfarin_miss <- produce_NA(warfarin_pk_baseline,
                             mechanism = "MAR",
                             perc.missing = 0.0625,
                             idx.incomplete = c(0, 0, 0, 0, 0, 0, 0, 1, 0, 0),
                             # create missing WT from observed age and sex
                             idx.covariates = c(0, 0, 0, 0, 0, 0, 0, 0, 1, 1),
                             weights.covariates = c(0, 0, 0, 0, 0, 0, 0, 0, 1/2, 1/2),
                             logit.model = "RIGHT",
                             seed = counter)

  # extract the generated incomplete dataset
  warfarin_incomp <- warfarin_miss$data.incomp

  # extract the eligible seeds
  if (sum(is.na(warfarin_incomp$WT)) == 2) { #6.25% WT missing corresponds to 2 patients
    # if a seed results in 6.25% WT missing, save that seed to the seeds_with_sum_2 vector
    seeds_with_sum_2 <- c(seeds_with_sum_2, counter)
  }
  # otherwise, increase counter by 1
  counter <- counter + 1
}
# The resulting seeds_with_sum_2 should contain 30 eligible seeds

## Generate missing datasets -----
# The next step is to plug in the seeds to have 30 different datasets with 6.25% WT missing.
# Here we selected the first seed for demonstration purposes.

# impute only using the baseline WT
warfarin_miss <- produce_NA(warfarin_pk_baseline,
                           mechanism = "MAR",
                           perc.missing = 0.0625,
                           idx.incomplete = c(0, 0, 0, 0, 0, 0, 0, 1, 0, 0),
                           idx.covariates = c(0, 0, 0, 0, 0, 0, 0, 0, 1, 1),
                           weights.covariates = c(0, 0, 0, 0, 0, 0, 0, 0, 1/2, 1/2),
                           logit.model = "RIGHT",
                           seed = seeds_with_sum_2[1]) # plug in the first seed from above

# extract the generated incomplete dataset
warfarin_incomp <- warfarin_miss$data.incomp

# match the missing WT to the long format
warfarin_incomp_long <- warfarin_pk
warfarin_incomp_long$WT <- ifelse(warfarin_incomp_long$ID %in% warfarin_incomp$ID,
                                warfarin_incomp$WT[match(warfarin_incomp_long$ID,
                                                         warfarin_incomp$ID)],
                                warfarin_incomp_long$WT)

# Multiple imputation - 1st seed -----

```

```

## PK parameters as predictors -----

# merge warfarin_incomp to base.individual.estimate for multiple imputation
warfarin_mi <- base.individual.estimate |>
  left_join(warfarin_incomp |> select(ID, WT, SEX, AGE), by = "ID")

# change the imputation method to pmm to avoid negative imputed WT
imp0 <- mice(warfarin_mi, maxit = 0)
meth <- imp0$method
meth["WT"] <- "pmm"

# do not use ID to impute WT
predictorMatrix <- imp0$predictorMatrix
predictorMatrix[, "ID"] <- 0

# set the number of iterations and imputations
maxit <- 20
nimp <- 20 # 20 imputations for 6.25% case

# identify all IDs with missing WT
missing_wt_ids <- warfarin_incomp |>
  filter(is.na(WT)) |>
  pull(ID)

# for each missing dataset, we performed multiple imputation with 30 different seeds
for (i in seq_len(30)) {

  imputed_data <- mice(data = warfarin_mi,
    method = meth,
    predictorMatrix = predictorMatrix,
    maxit = maxit,
    m = nimp,
    printFlag = FALSE,
    seed = nseed[i]) # seeds are set from 1 to 30

# each multiple imputation consists of 20 imputations
for (j in seq_len(nimp)) {
  # extract each multiply imputed dataset
  dat_MI <- complete(imputed_data, j)

  # extract the imputed WT
  dat_MI_filtered <- dat_MI |>
    filter(ID %in% missing_wt_ids)

  # fill the imputed WT to the full missing dataset
  dat_MI <- warfarin_incomp_long |>
    mutate(WT = ifelse(ID %in% missing_wt_ids,
      dat_MI_filtered$WT[match(ID,
        dat_MI_filtered$ID)],
      WT),
    lnWT = log(WT / 70)) # also add the centered WT

# fit the full model to the multiply imputed dataset and
# assign the fixed parameter estimates to an output file
assign(paste0("fit.war.mi", sprintf("%02d", j), ".seed",
  sprintf("%02d", i), ".6.25.", sprintf("%02d", 1)),
  nlmixr(complete.model.war.1cpt.ode,
    dat_MI,
    est = "focei",
    control = list(print = 0),

```

```

        foceiControl = list(outerOpt = "bobyqa",
                             maxInnerIterations = 500))$parFixed)
    }
}

## Time and concentrations as predictors -----

imp0 <- mice(warfarin_incomp_long, maxit = 0)

# method pmm for WT
meth <- imp0$method
meth["WT"] <- "pmm"

# remove redundant variables from the predictor matrix
predictorMatrix <- imp0$predictorMatrix
predictorMatrix[, "EVID"] <- 0
predictorMatrix[, "ID"] <- 0
maxit <- 20
# predictors include: time+conc+amt+sex+age

missing_wt_ids <- warfarin_incomp %>%
  filter(is.na(WT)) %>%
  pull(ID)

# for each missing dataset, we performed multiple imputation with 30 different seeds
for (i in seq_len(30)) {

  imputed_data <- mice(data = warfarin_incomp_long,
                       method = meth,
                       predictorMatrix = predictorMatrix,
                       maxit = maxit,
                       m = nimp,
                       printFlag = FALSE,
                       seed = nseed[i]) # seeds are set from 1 to 30

# each multiple imputation consists of 20 imputations
for (j in seq_len(nimp)) {
  # extract each multiply imputed dataset
  dat_MI <- complete(imputed_data, j)

# average WT per ID
average_wt_MI <- dat_MI %>%
  filter(ID %in% missing_wt_ids) %>%
  group_by(ID) %>%
  summarize(avg_WT = mean(WT, na.rm = TRUE))

dat_MI <- dat_MI %>%
  left_join(average_wt_MI, by = "ID") %>%
  mutate(WT = ifelse(ID %in% missing_wt_ids, avg_WT, WT),
         lnWT = log(WT / 70)) %>%
  select(-avg_WT)

# fit the full model to the multiply imputed dataset and
# assign the fixed parameter estimates to an output file
assign(paste0("fit.war.mi", sprintf("%02d", j), ".seed",
                                       sprintf("%02d", i), ".6.25.", sprintf("%02d", 1)),
       nlmixr(complete.model.war.1cpt.ode,
              dat_MI,
              est = "focei",
              control = list(print = 0),
              foceiControl = list(outerOpt = "bobyqa",

```

```

        maxInnerIterations = 500))$parFixed)
  }
}

## Extract the parameter estimate datasets -----

# create lists to store output datasets
combined_data_list_mi_converged <- list()
combined_data_list_mi_unconverged <- list()

# loop over all the output datasets
for (i in seq_len(30)) {
  for (j in seq_len(nimp)) {
    dataset_name <- paste0("fit.war.mi", sprintf("%02d", j),
                          ".seed", sprintf("%02d", i), ".6.25.",
                          sprintf("%02d", 1))

    if (exists(dataset_name) && !is.null(get(dataset_name))) {
      data <- get(dataset_name)

      # if there's a SE column, the model converged
      if ("SE" %in% colnames(data)) {
        extracted_data <- data[1:5, ] |>
          mutate(imputation = j,
                 later_seed = i,
                 initial_seed = 1,
                 type = "MI",
                 converged = "Yes")

        combined_data_list_mi_converged[[length(combined_data_list_mi_converged) + 1]] <- extracted_data
      }

      # if there's no SE column, the model did not converge
    } else {
      extracted_data <- data[1:5, ] |>
        mutate(imputation = j,
               later_seed = i,
               initial_seed = 1,
               type = "MI",
               converged = "No")

      combined_data_list_mi_unconverged[[length(combined_data_list_mi_unconverged) + 1]] <- extracted_data
    }
  }
}

# combine all the converged and unconverged datasets
merged.war.mi.seed01.30.6.25.01.converged <- do.call(rbind, combined_data_list_mi_converged)
merged.war.mi.seed01.30.6.25.01.unconverged <- do.call(rbind, combined_data_list_mi_unconverged)

# Median imputation -----
# Here we also used the first missing dataset for demonstration purposes.

# extract the observed median value then use it for imputation
median_WT <- median(warfarin_incomp$WT, na.rm = TRUE)

warfarin_median <- warfarin_incomp_long |>
  mutate(WT = ifelse(is.na(WT), median_WT, WT))
# warfarin_incomp_long was created above

# center WT around 70kg

```

```
warfarin_median$lnWT <- log(warfarin_median$WT/70)

# fit the full model to the median imputed dataset
# assign the fixed parameter estimates to an output file
assign(paste0("fit.war.median.6.25.", sprintf("%02d", 01)),
      nlmixr(complete.model.war.1cpt.ode,
            warfarin_median,
            est = "focei",
            control = list(print = 0))$parFixed)
```

4 Session details

```
getS3method("print", "sessionInfo")(sessionInfo()[7])
```

```
R version 4.3.1 (2023-06-16 ucrt)
Platform: x86_64-w64-mingw32/x64 (64-bit)
Running under: Windows 11 x64 (build 22631)
```

```
Matrix products: default
```

```
Random number generation:
```

```
RNG:
Normal:
Sample:
```

```
locale:
```

```
[1] LC_COLLATE=English_United Kingdom.utf8
[2] LC_CTYPE=English_United Kingdom.utf8
[3] LC_MONETARY=English_United Kingdom.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United Kingdom.utf8
```

```
time zone: Europe/Brussels
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] parallel splines stats graphics grDevices utils datasets
[8] methods base
```

```
other attached packages:
```

```
[1] gdata_3.0.0 devtools_2.4.5
[3] usethis_3.0.0 nlmixr2_3.0.1
[5] nlmixr2data_2.0.9 lubridate_1.9.3
[7] forcats_1.0.0 stringr_1.5.1
[9] purrr_1.0.2 readr_2.1.5
[11] tidyr_1.3.0 tibble_3.2.1
[13] ggplot2_3.5.1 tidyverse_2.0.0
[15] countimp_2.0.7 pscl_1.5.9
[17] MASS_7.3-60 aster_1.1-3
[19] glmmTMB_1.1.10 ImputeRobust_1.3-1
[21] gamlss_5.4-22 nlme_3.1-165
[23] gamlss.dist_6.1-1 gamlss.data_6.0-6
[25] mice_3.16.0 dplyr_1.1.3
[27] packagemetrics_0.0.1.9001
```

```
loaded via a namespace (and not attached):
```

```
[1] sys_3.4.2 rstudioapi_0.16.0 jsonlite_1.8.7
[4] shape_1.4.6.1 magrittr_2.0.3 lotri_0.4.3
[7] jomo_2.7-6 nloptr_2.1.1 rmarkdown_2.27
```

[10]	fs_1.6.3	PreciseSums_0.6	vctr_0.6.3
[13]	memoise_2.0.1	minqa_1.2.7	janitor_2.2.1
[16]	htmltools_0.5.8.1	curl_5.1.0	broom_1.0.6
[19]	rxode2et_2.0.13	mitml_0.4-5	htmlwidgets_1.6.4
[22]	cachem_1.1.0	TMB_1.9.16	mime_0.12
[25]	lifecycle_1.0.4	iterators_1.0.14	pkgconfig_2.0.3
[28]	Matrix_1.6-4	R6_2.5.1	fastmap_1.2.0
[31]	shiny_1.9.1	rbibutils_2.3	snakecase_0.11.1
[34]	digest_0.6.33	selectr_0.4-2	numDeriv_2016.8-1.1
[37]	colorspace_2.1-0	ps_1.7.5	pkgload_1.4.0
[40]	extremevalues_2.4.1	fansi_1.0.4	timechange_0.2.0
[43]	httr_1.4.7	mgcv_1.9-1	compiler_4.3.1
[46]	remotes_2.5.0	withr_3.0.1	backports_1.5.0
[49]	qs_0.26.3	pkgbuild_1.4.4	pan_1.9
[52]	nlmixr2plot_2.0.9	n1qn1_6.0.1-11	sessioninfo_1.2.2
[55]	gtools_3.9.5	dparser_1.3.1-11	tools_4.3.1
[58]	chromote_0.4.0	trust_0.1-8	httpuv_1.6.12
[61]	rxode2random_2.1.1	nnet_7.3-19	glue_1.6.2
[64]	lbfgsb3c_2024-3.4	promises_1.2.1	grid_4.3.1
[67]	checkmate_2.3.2	rxode2_2.1.3	generics_0.1.3
[70]	gtable_0.3.5	tzdb_0.4.0	websocket_1.4.2
[73]	data.table_1.15.4	RApiSerialize_0.1.3	hms_1.1.3
[76]	xml2_1.3.6	stringfish_0.16.0	utf8_1.2.3
[79]	foreach_1.5.2	pillar_1.9.0	later_1.3.1
[82]	lattice_0.22-6	survival_3.7-0	tidyselect_1.2.1
[85]	miniUI_0.1.1.1	knitr_1.48	symengine_0.2.6
[88]	reformulas_0.4.0	cranlogs_2.1.1	xfun_0.46
[91]	stringi_1.7.12	yaml_2.3.7	boot_1.3-28.1
[94]	evaluate_0.24.0	codetools_0.2-19	nlmixr2est_2.2.2
[97]	cli_3.6.1	RcppParallel_5.1.8	rpart_4.1.23
[100]	xtable_1.8-4	Rdpack_2.6.2	munsell_0.5.1
[103]	processx_3.8.3	Rcpp_1.0.11	ellipsis_0.3.2
[106]	profvis_0.3.8	urlchecker_1.0.1	lme4_1.1-35.5
[109]	glmnet_4.1-8	scales_1.3.0	rxode2parse_2.0.19
[112]	rlang_1.1.1	rvest_1.0.4	